Waste Sorting and Recycling Assistant

Project and home-task presentation guide

Purpose

This guide explains the capstone application, the four subsystems, the four independent home-task solutions, and the exact responsibility of each team member. The project uses a local YOLO image model for recognition and a configurable OpenAI-compatible API for natural-language recycling guidance. Codex-specific runtime logic is not part of the application.

System flow

Upload image -> validate and preprocess -> local YOLO recognition -> confidence filtering -> recycling rule lookup -> LLM answer or local fallback -> Flask dashboard

The visual model remains local. The API key is supplied at runtime with --api-key or the LLM_API_KEY environment variable and is never committed to Git.

Running the main project

```
python -m pip install -r requirements.txt
$env:LLM_API_KEY="your-key"
python main.py --model models/best_model.pt
```

Team responsibilities

Team member	Subsystem	Main classes	Presentation focus
1	Dataset and preprocessing	ImageLoader, ImageValidator, PreprocessingPipeline, DatasetStatistics	Input safety, format validation, orientation correction, resizing, and dataset metadata.
2	AI image recognition	WasteDetector, Detection, BoundingBox, ConfidenceFilter, DetectionStream	Model loading, inference, confidence filtering, box data, and iterator-based stream processing.
3	Recycling rules and Ask AI	RecyclingRule, RecyclingRuleEngine, PromptBuilder, OpenAICompatibleClient, WasteAssistant	Local rules, runtime API key, prompt design, fallback behavior, and concise answers.
4	Web dashboard	Flask app, routes, request validation, error responses, UI integration	How the upload and Ask AI routes connect all subsystems.

Home Task 1 - UML and classes

Use home task 1 - uml and classes. Run python main.py. Explain the four subsystem boundaries, class ownership, composition, and the capstone UML Mermaid file. Every team member has at least three unique classes.

Home Task 2 - OOP concepts

Use home task 2 - oop concepts. Run python main.py. Demonstrate Processor as an ABC, polymorphic subclasses, the | operator, the generator stream, confidence filtering, box normalization, and label-to-material transformation.

Home Task 3 - Flask and MySQL

Use home task 3 - flask mysql. Run python app.py. Explain GET /, POST /records, GET /records, validation, the SQL schema, and how DATABASE_URL can connect to MySQL. The default in-memory list runs immediately.

Home Task 4 - exception handling and tests

Use home task 4 - exception handling and tests. Run python -m pytest -q. Explain one exception boundary per subsystem and the 15 tests. The tests do not need a model checkpoint or API key.

Suggested presentation sequence

1. Team member 4 opens the dashboard and explains the user journey. 2. Team member 1 explains validation and preprocessing. 3. Team member 2 demonstrates recognition and confidence filtering. 4. Team member 3 explains local rules, the API-based Ask AI response, runtime keys, and fallback. 5. The team runs the 15 tests and opens the UML diagram. 6. Close by showing that each home task runs independently.

Assessment checklist

Requirement	Evidence
Three unique classes per student	Class ownership table and Home Task 1 demo
ABCs, inheritance, polymorphism	Home Task 2 Processor hierarchy
Operator overloading	Processor.__or__ pipeline composition
Iterators and generators	DetectionStream and stream() examples
Flask and MySQL	Main dashboard, independent Flask sample, schema.sql
Exception handling and 15 tests	Subsystem exceptions and Home Task 4 test suite